

K230 Gesture Recognition

K230 Gesture Recognition

K230 and MSPM0 Communication

1. Experimental Prerequisites
2. Experimental Wiring
3. Main Code Explanation
4. Experimental Phenomenon

K230 and MSPM0 Communication

1. Experimental Prerequisites

This tutorial uses the MSPM0G3519 development board. The corresponding example path is 【14.export\MSPM0-K230\12_k230_gesture】.

K230 needs to run the 【14.export\CanmvIDE-K230\12.gesture.py】 program to start the experiment. It is recommended to download as an offline running program.

Items needed:

Windows system computer

MSPM0G3519 development board

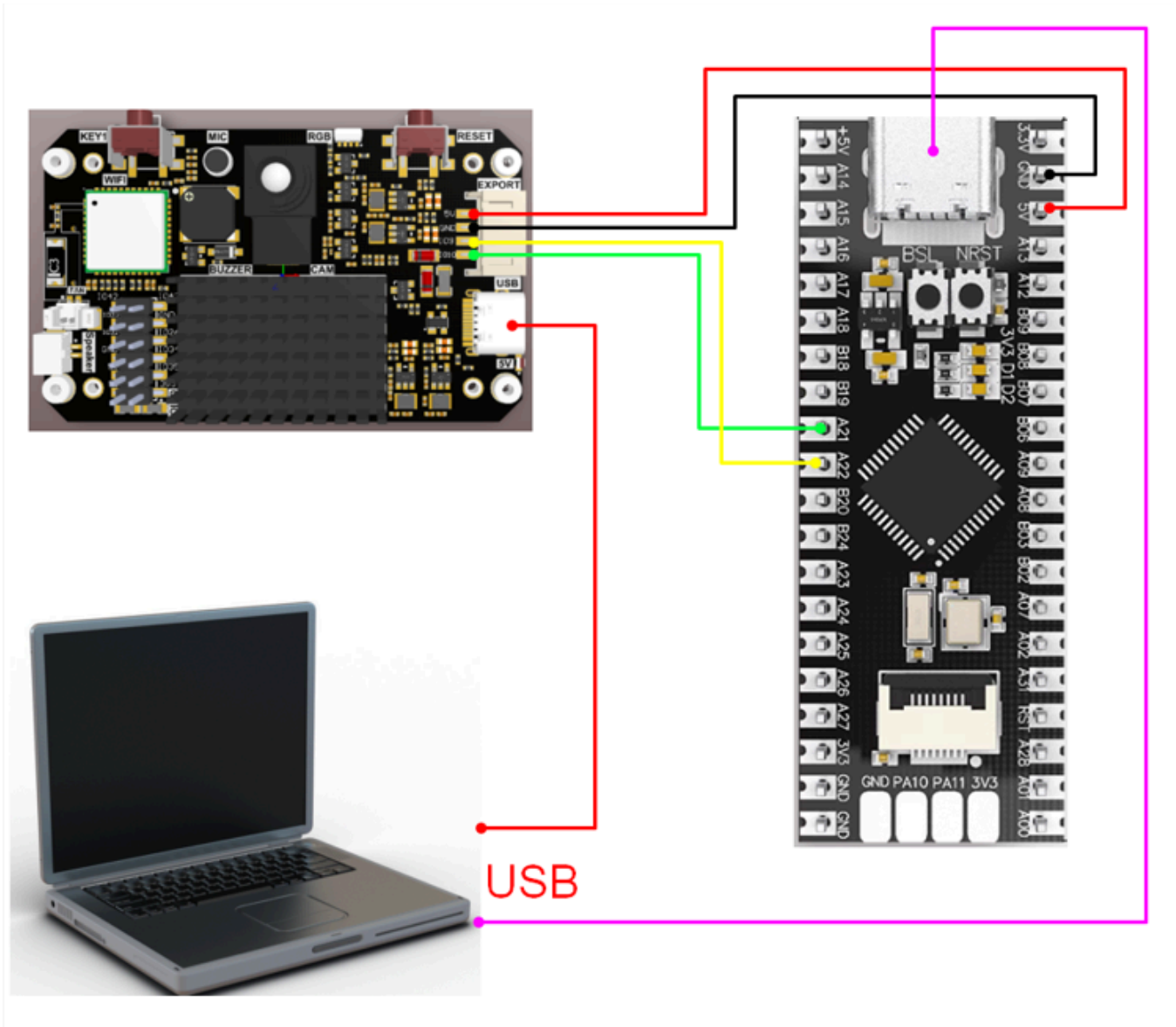
K230 vision module (with TF card burned with image)

Two type-C data cables

Connecting cables

2. Experimental Wiring

K230 Vision Module	MSPM0 Development Board
5V	VCC
GND	GND
TXD(IO9)	A22
RXD(IO10)	A21



3. Main Code Explanation

```
void Pto_Data_Parse(uint8_t *data_buf, uint8_t num)
{
    uint8_t pto_head = data_buf[0];
    uint8_t pto_tail = data_buf[num-1];
    if (!(pto_head == PTO_HEAD && pto_tail == PTO_TAIL))
    {
        sprintf(print_buf, "pto error:pto_head=0x%02x , pto_tail=0x%02x\n", pto_head,
pto_tail);
        uart0_send_string(print_buf);
        return;
    }
    uint8_t data_index = 1;
    uint8_t field_index[PTO_BUF_LEN_MAX] = {0};
    int i = 0;
    int values[PTO_BUF_LEN_MAX] = {0};
    for (i = 1; i < num-1; i++)
    {
        if (data_buf[i] == ',')
```

```

    {
        data_buf[i] = 0;
        field_index[data_index] = i;
        data_index++;
    }
}
char msg[PTO_BUF_LEN_MAX] = {0};
for (i = 0; i < data_index; i++)
{
    if (i == 2)
    {
        memcpy(msg, (char*)data_buf+field_index[i]+1, values[0]-field_index[i]-2);
    }
    else
    {
        values[i] = Pto_Char_To_Int((char*)data_buf+field_index[i]+1);
    }
}

uint8_t pto_len = values[0];

if (pto_len != num)
{
    sprintf(print_buf, "pto_len error:%d , data_len:%d\n", pto_len, num);
    uart0_send_string(print_buf);
    return;
}
uint8_t pto_id = values[1];
if (pto_id != PTO_FUNC_ID)
{
    sprintf(print_buf, "pto_id error:%d, func_id:%d\n", pto_id, PTO_FUNC_ID);
    uart0_send_string(print_buf);
    return;
}

sprintf(print_buf, "gesture: '%s'\n", msg);
uart0_send_string(print_buf);
}

```

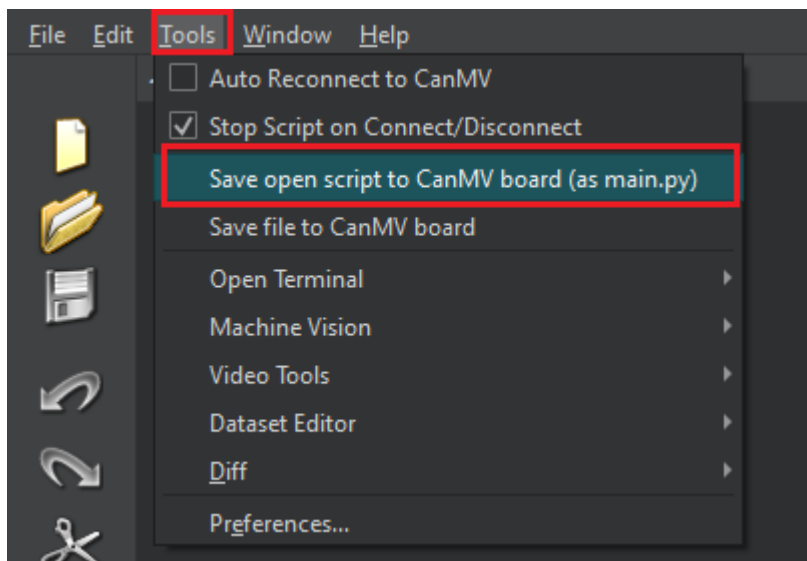
The above function is for parsing K230 data. Only data conforming to the specific protocol can be parsed to get the corresponding data.

Where:

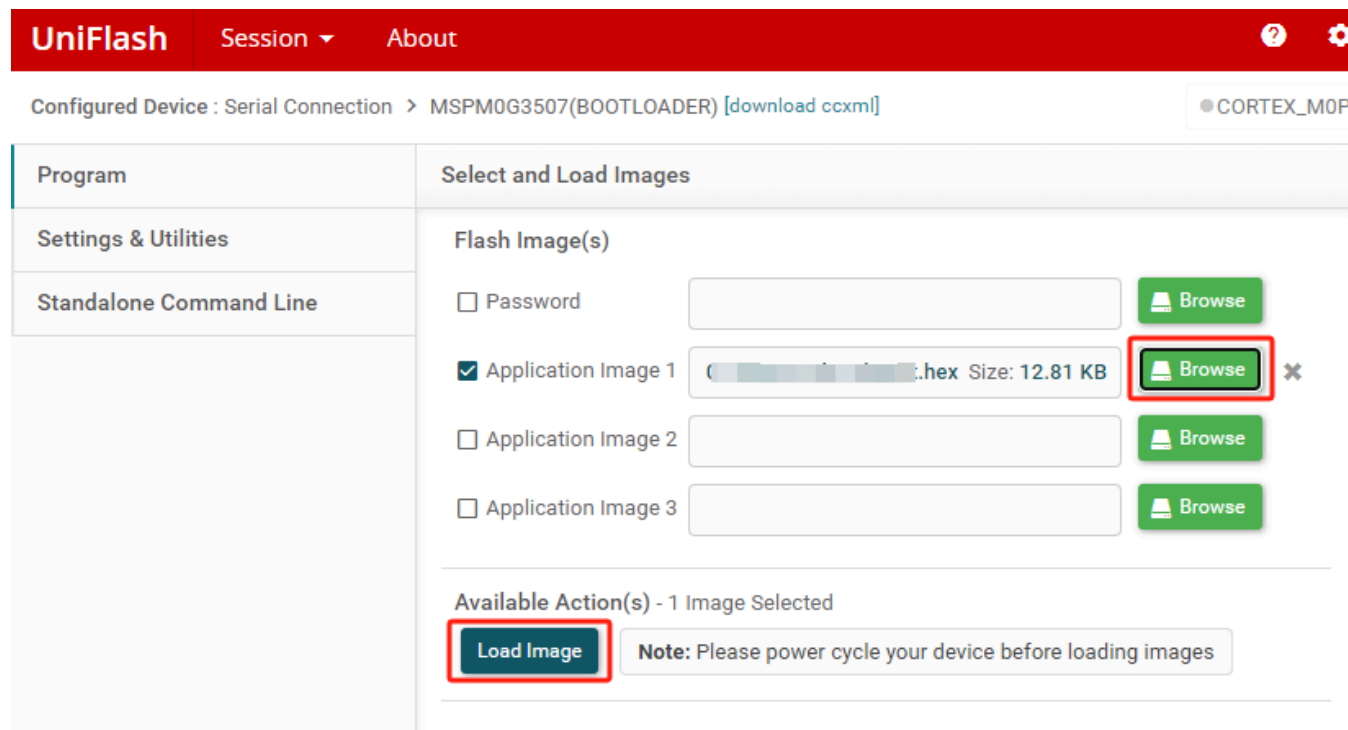
- gesture: msg is the gesture information. 'UP' for up, 'DOWN' for down, 'LEFT' for left, 'RIGHT' for right, 'MIDDLE' for center

4. Experimental Phenomenon

1. After connecting the cables, the K230 vision module runs offline
Connect K230 to Canmv IDE, open the corresponding program, click 【Save open script to CanMV board (as main.py)】 in the toolbar, then restart K230.



2. MSPM0 burns the hex file generated by the example.



3. Set the serial assistant interface as shown in the figure

Port

COM25:USB 串行设备

Baud rate 115200

Stop bits 1

Data bits 8

Parity None

Operation  Close

Save Data Clear Data

☐ Hex ☐ DTR

☐ RTS ☐ 自动保存

☐ TimeStamp 1000 ms

4. When the K230 camera image recognizes a palm and the palm makes a corresponding gesture, the serial assistant will print the information transmitted by K230 to MSPM0.

Where:

- gesture: is the gesture information. 'UP' for up, 'DOWN' for down, 'LEFT' for left, 'RIGHT' for right, 'MIDDLE' for center

As shown in the figure below

```
[2025-04-30 12:02:09.936]# RECV ASCII>
gesture:'DOWN'

[2025-04-30 12:02:13.937]# RECV ASCII>
gesture:'UP'

[2025-04-30 12:02:19.139]# RECV ASCII>
gesture:'RIGHT'

[2025-04-30 12:02:21.556]# RECV ASCII>
gesture:'RIGHT'

[2025-04-30 12:02:22.700]# RECV ASCII>
gesture:'RIGHT'
```